1. Problem Title & Link

Title: **Toggle Bits of a Decimal Number**

(Source: TCS NQT Previous Coding Questions PDF)

2. Problem Statement (Short Summary)

You are given a **positive integer N**.

Steps to perform:

1. Convert the number to its **binary representation**.
2. **Toggle all bits** of the binary number.
   - 0 → 1
   - 1 → 0
3. Convert the resulting binary number **back to decimal**.

Return the resulting decimal number.

3. Examples

Example 1

Input

10

Output

5

Explanation

10 → binary = 1010

toggle → 0101

decimal = 5

Example 2

Input

7

Output

0

Explanation

7 → binary = 111

toggle → 000

decimal = 0

4. Constraints

$1 \le N \le 100$

Observations

- Maximum binary length ≈ **7 bits**
- Bit manipulation is straightforward

5. Core Concept (Pattern)

Bit Manipulation

Concepts used:

Binary representation

Bit toggle

Bit mask

6. Thought Process

Step 1 – Convert to Binary

Example

10 → 1010

Step 2 – Toggle Each Bit

1 → 0

0 → 1

Result

1010 → 0101

Step 3 – Convert Back to Decimal

0101 → 5

Important Observation

If a number has **k bits**, the mask of all ones is:

(2^k − 1)

Example

10 = 1010 (4 bits)

mask = 1111

Then toggling becomes

result = mask XOR number

7. Visualization

Example

N = 10

Binary

1010

Mask

1111

XOR

1010

1111

0101

Decimal

5

8. Pseudocode

```
read N

find number of bits in N

mask = (1 << bits) - 1

result = mask XOR N

print result
```

9. Code Implementation

Python

```
n = int(input())

bits = n.bit_length()

mask = (1 << bits) - 1

result = mask ^ n

print(result)
```

Java

```
import java.util.*;

class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int bits = Integer.toBinaryString(n).length();

        int mask = (1 << bits) - 1;

        int result = mask ^ n;

        System.out.println(result);
    }
}
```

10. Time & Space Complexity

Time Complexity

O(log N)

Because the number of bits is proportional to log N.

Space Complexity

O(1)

Only a few variables are used.

11. Common Mistakes

1. Forgetting to restrict toggle to **existing bits**
2. Using bitwise NOT ~n (which affects infinite bits)
3. Incorrect mask calculation
4. Leading zero confusion

Example mistake

~10 = -11 ❌

12. Detailed Dry Run

Example

N = 10

Binary

1010

Bit length

4

Mask

(1<<4)-1

= 16-1

= 15

= 1111

XOR

1010

1111

0101

Result

5

13. Real-Life Applications

Bit manipulation appears in:

- Encryption algorithms
- Network packet encoding
- Hardware circuit design
- Memory optimization

14. Common Traps

1. Using incorrect bit count
2. Not converting binary correctly
3. Using negative bit operations
4. Ignoring leading zeros

15. Related Problems

Bit manipulation practice:

LC 190 Reverse Bits

LC 191 Number of 1 Bits

LC 476 Number Complement

LC 1009 Complement of Base 10 Integer

16. Alternate Approaches

Approach	Complexity	Notes
String binary toggle	O(log N)	simple but slower
Bit mask XOR	O(log N)	optimal

17. Why This Solution Works

The mask

(2^bits − 1)

creates a number with **all bits set to 1**.

XOR with the original number flips every bit:

1 XOR 1 = 0

0 XOR 1 = 1

Thus effectively **toggling all bits**.

18. Variations / Follow-Up Questions

Possible interview follow-ups:

1. Toggle **only even bit positions**
2. Toggle **only odd bit positions**
3. Toggle bits **between L and R**
4. Find complement of a number